Module 1

Module Title: Introduction to Programming Languages

Module Description:

This module motivates the study of programming languages, introduces the major language families, and provides an overview of the compilation process.

Purpose of the Module:

This module let the students learn the importance of :

- Studying programming Language and Preliminaries and Evolution of Programming Language
- Syntax and Semantics, Lexical and Syntax Analysis, or textual structure, of programs
- High-level structure of programs.
- How a compiler (or interpreter) determines the semantics, or meaning of a program.

Module Guide:

Module 1 Set the stage by covering foundational material in both design and implementation.

Module Outcomes:

1. Describe the importance of Language and Preliminaries, and Evolution of Programming Language. Also, Learn the significance of Syntax and Semantics, and Lexical and Syntax Analysis
2. Evaluate the appropriateness of the use of a programming language for implementing a particular application based on language features.

Module Requirements:

At the end of this module, the students will come up with a comparative study of different programming paradigms.

.

Module Pre-test:

Answer the following questions:

1. What is the Purpose of Language?
2. What is the meaning of Plankalkül? Who developed Plankalkül?
3. Which is the first successful high-level programming language for business?
4. How did Plankalkül implement iterations?
5. Why was the technique used by Short Code called automatic programming?
6. How are programming languages formally defined?
7. What is a metalanguage?
8. What are the reasons why using BNF is advantageous over using an informal syntax description?
9. Define finite automata and regular grammar.

Learning Plan

Lesson No: 01

Lesson Title: Language and Preliminaries and Its Evolution of the Major Programming Language

Let's Hit These:

At the end of this lesson, students should be able to:

- set the stage by covering foundational material in both design and implementation.
- motivates the study of programming languages,
- introduces the major language families, and provides an overview of the compilation process.

Let's Get Started:

1. Why is it useful for a programmer to have the ability to learn new languages, even though he or she may have a good knowledge of a number of programming languages?
2. Why is it essential to choose an appropriate programming language for a specific software solution?
3. Which programming language for scientific applications was the first to be used successfully?
4. Which is the first successful high-level programming language for business?
5. In which programming language were most of the AI applications developed prior to 1990?
6. What is the meaning of Plankalkül? Who developed Plankalkül?
7. What was the simplest data type in Plankalkül?
8. How did Plankalkül implement iterations?
9. Speedcoding was invented to overcome two significant shortcomings of the computer hardware of the early 1950s. What were they?
10. Enumerate the Hierarchy of Programming?

Let's Find Out:

- Why there are so many languages?
- What makes them successful or unsuccessful?
- What is the difference between language design and language interpretation

Let's Read:

✓ Language

What is a Language?

- Spoken by the individuals
- A medium of communication
- A system of signs and symbols and rules for using them that is used to carry information.
- A system for communicating. Written languages use symbols (that is, characters) to build words. The entire set of words is the language's *vocabulary*. The ways in which the words can be meaningfully combined are defined by the language's *syntax* and *grammar*. The actual meaning of words and combinations of words is defined by the language's semantics.

  In computer science, human languages are known as *natural language*. Unfortunately, computers are not sophisticated enough to understand natural languages. As a result, we must communicate with computers using special computer languages.

  There are many different classes of computer languages, including:

  a) Machine languages
  b) Programming languages
  c) Fourth-generation languages.

- Machine languages – The lowest-level programming language. Machine languages are the only languages understood by computers. While easily understood by computers, machine languages are almost impossible for humans to use because they consist entirely of numbers. Programmers, therefore, use either a *high-level programming language* or an *assembly language*. An assembly language contains the same instructions as a machine language, but the instructions and variables have names instead of being just numbers.
- Programming languages – A vocabulary and set of grammatical rules for instructing a computer to perform specific tasks. The term *programming language* usually refers to *high-level* languages, such as BASIC, C, COBOL, FORTRAN, and Pascal. Each language has unique set of keywords (words that it understands) and a special syntax for organizing program instructions.

  High-level programming languages, while simple compared to human languages, are more complex that the languages the computer actually understands, called *machine languages*. Each different type of CPU has its own unique machine language.

  Lying between machine languages and high-level languages are languages called *assembly languages* (Figure 1). Assembly languages are similar to machine language, but they are much easier to program in because they allow a programmer to substitute names for numbers. Machine languages consist numbers only.

  Lying above high-level languages are language called *fourth-generation languages* (usually abbreviated 4GL). 4GLs are far removed from machine languages and represent the class of computer languages closest to human languages.

  Regardless of what language you use, eventually you need to convert your program into machine language so that the computer can understand it.

There are two ways to do this:

a) *Compile* the program
b) *Interpret* the program

COMPILER VS. INTERPRETER

- Both translators

Differences

A compiler is a translator program that reads a program (source) and then translates it into a program written into another language.

Compiler – A program that translates *source code* into *object code*. The compiler derives its names from the way it works, looking at the entire piece of source code and collecting and reorganizing the instructions.

Interpreter – A program that executes instructions written in a high level language. There are two ways to run programs written in a high level language. The most common is to *compile* the program; the other method is to pass the program through an interpreter.

An interpreter translates high level-level instructions into an intermediate form, which it then executes. In contrast, a compiler translates high-level instructions directly into machine language. Compiled programs run much faster than interpreted programs.

Generations of Computer Languages

Fourth-generation languages – Often abbreviated *4GL,* fourth-generation languages are programming languages closer to human languages than typical high-level programming languages (see Figure 1). Most 4GLs are used to access databases. For example, typical 4GL command is

FIND ALL RECORDS WHERE NAME IS "SMITH"

The other three generations of computer languages are:

First generation: Machine language

@ First Generation 1940's & 1950's

- All coding is done in machine language the computer's (binary – based) instruction set.

The first electronic computers were gigantic machine, filling several rooms, consuming as much electricity as a good-size factory, and costing millions money but with much less computing power than even the simplest modern cell phone today. The programmers who used these machines believed that the computer's time was more valuable than theirs. They programmed in machine language. A Machine language is the sequence of bits that directly controls a processor, causing it to add, compare, move data from one place to another, and so forth at appropriate times. Specifying programs at this level of detail is an enormously tedious task. The following program calculates the greatest common divisor (GCD) of two integers, using Euclid's algorithm. It is written in machine language, expressed here as hexadecimal (base 16) numbers, for the x86 instruction set.

```
55 89 e5 53   83 ec 04 83   e4 f0 e8 31   00 00 00 89   c3 e8 2a 00
00 00 39 c3   74 10 8d b6   00 00 00 00   39 c3 7e 13   29 c3 39 c3
75 f6 89 1c   24 e8 6e 00   00 00 8b 5d   fc c9 c3 29   d8 eb eb 90
```

Example 1.1 GCD program in x86 machine language

Second generation: Assembly language

@ Second Generation

- Development & distribution of machine – code subroutines, interpretive routines, automatic code generator and assemblers.

As people began to write larger programs, it quickly became apparent that a less error-prone notation was required. Assembly languages were invented to be expressed with mnemonic abbreviations. Our GCD program looks like this in x86 assembly language:

```
        pushl   %ebp                    jle     D
        movl    %esp, %ebp              subl    %eax, %ebx
        pushl   %ebx            B:      cmpl    %eax, %ebx
        subl    $4, %esp                jne     A
        andl    $-16, %esp      C:      movl    %ebx, (%esp)
        call    getint                  call    putint
        movl    %eax, %ebx              movl    -4(%ebp), %ebx
        call    getint                  leave
        cmpl    %eax, %ebx              ret
        je      C               D: subl %ebx, %eax
A:      cmpl    %eax, %ebx              jmp B
```

Example 1.2 **GCD program in x86 assembler**

Third generation: High-level programming language

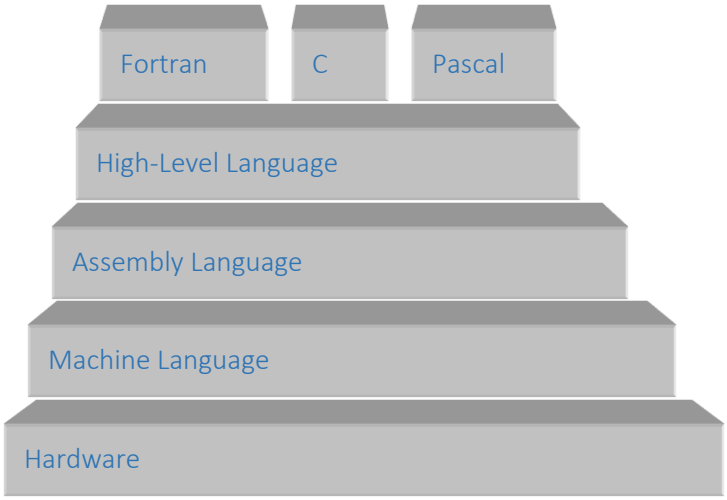


Figure 1: *Hierarchy of Programming Languages*

What's the purpose of language?

- For interaction
- For the exchange of information
- For expression of one's ideas, thoughts, feelings.

How is a Language formed?

- It is a notation for writing programs.
  (notation) The act, process or method of representing dot by marks, signs, symbols, figure, or characters.

@ GENERAL Criteria for Classifying a Programming Language

- The degree for which they are defend on the underlying hardware configuration (low-level vs. high-level).
- The degree to which programs written in that language are processes oriented.
- The data type that is assumed or that the language handles most easily (data type).
- The problems it is designed to solve (problem oriented)
- The degree to which the language is extensible.
- The degree (any kind) of interaction of the program with its environment.
- The programming paradigm most naturally supported

@ CHARACTERISTIC of a Programming Language.

- Writability
- Error-checking
- Readability
- Self-documenting
- Extensibility
- Portabilty
- Efficiency

✓ PRELIMINARIES

- REASONS FOR STUDYING CONCEPTS OF PROGRAMMING LANGUAGES

1) INCREASED CAPACITY TO EXPRESS IDEAS

It is widely believed that the depth at which we can think is influenced by the expressive power of the language in which we communicate our thoughts. Those with a limited grasp of natural language are limited in the complexity of their thoughts, particularly in the depth of abstraction. In other words, it is difficult for people to conceptualize structures that they cannot describe, verbally or in writing.

2) IMPROVED BACKGROUND OF CHOOSING APPROPRIATE LANGUAGES

Many professional programmers have had little formal education in computer science and were trained on the job or through in-house training programs. Such training programs often teach one or two languages that are directly relevant to the current work of the organization.

3) INCREASED ABILITY TO LEARN NEW LANGUAGES

Computer programming is a young discipline, and design methodologies, software development tools, and programming languages are still in a state of continuous evolution.

4) BETTER UNDERSTANDING OF THE SIGNIFICANCE OF IMPLEMETATION

It is both interesting and necessary to touch on the implementations issues that affect those concepts. In some cases, an understanding of implementation issues leads to an understanding of why language is designed the way they are.

5) INCREASED ABILITY TO DESIGN NEW LANGUAGES

To a student, the possibility of being required at some future time to design a new programming language may seem remote.

6) OVERALL ADVANCEMENT OF COMPUTING

Finally, there is a global view of computing that can justify the study of programming language concepts. Although it is usually possible to determine why a particular programming language became popular, it is not always clear, at least in retrospect, that the most popular languages are the best available. In some cases, it might be concluded, a language became widely used, at least in part, because those in positions to choose languages were not sufficiently familiar with programming language concepts.

- PROGRAMMING DOMAINS

1) SCIENTIFIC APPLICATION

The first digital computers, which appeared in the 1940's, were used and in fact invented for scientific applications. Typically, scientific applications have simple data structures but require large numbers of floating-point arithmetic computations. The first language for scientific applications was FORTRAN.

2) BUSINESS APPLICATION

The use of computers for business applications began in the 1950's. Special computers for this purpose were developed, along with special language. The first successful high-level language for business was COBOL (ANSI, 1985), which appeared in 1960.

3) ARTIFICIAL INTELLIGENCE

Artificial Intelligence (AI) is a broad area of computer applications characterized by the absence of exact algorithms and the use of symbolic computation rather than numeric computation. Symbolic computations means that symbols, consisting of names rather than numbers, are manipulated. Also, symbolic computation is more conveniently done with linked lists of data rather than arrays.

The first widely used programming language developed for AI applications was the functional language LISP (McCarthy et. al., 1965), which appeared in 1959.

4) SYSTEMS PROGRAMMING LANGUAGES

The operating system and all of the programming support tools of a computer system are collectively known as its **system software.** Systems software is used almost continuously and therefore must have execution efficiency. A language for this domain must have low-level features that allow the software interfaces to external devices to be written.

5) VERY HIGH-LEVEL LANGUAGES (VHLLs)

The languages in the category called very high-level have evolved slowly over the past 25 years. The various scripting languages for UNIX are examples of VHLLs. A scripting language is one that is used by putting a list of commands, called a script, in a file to be executed. The first of these languages, named **shell**, began a small collection of commands that were interpreted to be calls to system subprogram that perform utility functions, such as file management and simple file filtering.

6) SPECIAL-PURPOSE LANGUAGES

A host of special-purpose languages have appeared over the past 40 years. They range from RPG, which is used to produce business reports, to APT, which is used for instructing programmable machine tools, to GPSS, which is used for system simulation.

LANGUAGE EVALUATION CRITERIA (CHARACTERISTICS OF PROG…)

We will also evaluate these features, focusing in their impact on the software development process. To accomplish this, we need a set of evaluation criteria. In spite of these differences, most computers scientists would agree that the criteria discussed in the following subsections are important.

Some of the factors that influence the most important of these criteria are shown in the following list, and the criteria themselves are discussed in the following sections.

	READABILITY	WRITABILITY	RELIABILITY
Syntax design	X		X
Control structures	X	X	X
Data types & structures	X	X	X
Simplicity/orthogonality	X	X	X
Support for abstraction		X	X
Expressivity		X	X
Type checking			X
Exception handling			X
Restricted Aliasing			X

Overall Simplicity

The overall simplicity of a programming language strongly affects its readability. A language with a large number of basic constructs is more difficult to learn than one with a smaller number. Programmers who must use a large language often learn a subset of the language and ignore its other features. This learning pattern is sometimes used to excuse the large number of language constructs, but that argument is not valid. Readability problems occur whenever the program's author has learned a different subset from that subset with which the reader is familiar. A second complicating

characteristic of a programming language is feature multiplicity— that is, having more than one way to accomplish a particular operation. For example, in Java, a user can increment a simple integer variable in four different ways:

count = count + 1

count += 1 count++

++coun

Although the last two statements have slightly different meanings from each other and from the others in some contexts, all of them have the same meaning when used as stand- alone expressions. These variations are discussed in Chapter 7. A third potential problem is operator overloading, in which a single operator symbol has more than one meaning. Although this is often useful, it can lead to reduced readability if users are allowed to create their own overloading and do not do it sensibly. For example, it is clearly acceptable to overload + to use it for both integer and floating-point addition. In fact, this overloading simplifies a language by reducing the number of operators. However, suppose the programmer defined + used between single- dimensioned array operands to mean the sum of all elements of both arrays. Because the usual meaning of vector addition is quite different from this, this unusual meaning could confuse both the author and the program's readers. An even more extreme example of program confusion would be a user defining + between two vector operands to mean the difference

Orthogonality

in a programming language means that a relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language. Furthermore, every possible combination of primitives is legal and meaningful. For example, consider data types. Suppose a language has four primitive data types (integer, float, double, and character) and two type operators (array and pointer). If the two type operators can be applied to themselves and the four primitive data types, a large number of data structures can be defined. The meaning of an orthogonal language feature is independent of the context of its appearance in a program. (The word orthogonal comes from the mathematical concept of orthogonal vectors, which are independent of each other.) Orthogonality follows from a symmetry of relationships among primitives. A lack of orthogonality leads to exceptions to the rules of the language. For example, in a programming language that supports pointers, it should be possible to define a pointer to point to any specific type defined in the language. However, if pointers are not allowed to point to arrays, many potentially useful user-defined data structures cannot be defined.

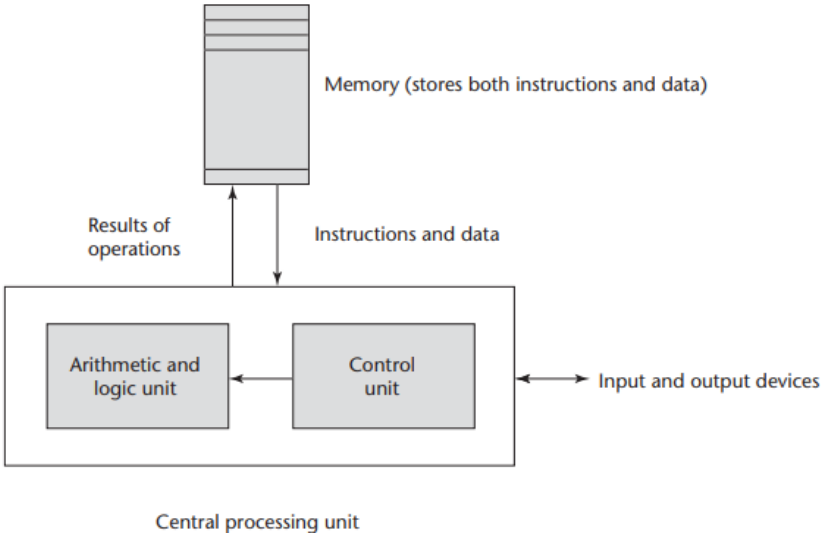
Data Types

The presence of adequate facilities for defining data types and data structures in a language is another significant aid to readability. For example, suppose a numeric type is used for an indicator flag because there is no Boolean type in the language. In such a language, we might have an assignment such as the following: timeOut = 1 The meaning of this statement is unclear, whereas in a language that includes Boolean types, we would have the following: timeOut = true The meaning of this statement is perfectly clear.

Influence in Language Design

Computer Architecture The basic architecture of computers has had a profound effect on language design. Most of the popular languages of the past 60 years have been designed around the prevalent computer architecture, called the von Neumann architecture, after one of its originators, John von Neumann (pronounced "von Noyman"). These languages are called imperative languages. In a von Neumann computer, both data and programs are stored in the same memory. The central processing unit (CPU), which executes instructions, is separate from the memory. Therefore, instructions and data must be transmitted, or piped, from memory to the CPU. Results of operations in the CPU must be moved back to memory. Nearly all digital computers built since the 1940s have been based on the von Neumann architecture. The overall structure of a von Neumann computer is shown in Figure 1.1. Because of the von Neumann architecture, the central features of imperative languages are variables, which model the memory cells; assignment statements, which are based on the piping operation; and the iterative form of repetition, which is the most efficient way to implement repetition on this architecture. Operands in expressions are piped from memory to the CPU, and the result of evaluating the expression is piped back to the memory cell represented by the left side of the assignment. Iteration is fast on von Neumann computers because instructions are stored in adjacent cells of memory and repeating the execution of a section of code requires only a branch instruction. This efficiency discourages the use of recursion for repetition, although recursion is sometimes more natural.

Figure 1.1
The von Neumann computer architecture



The execution of a machine code program on a von Neumann architecture computer occurs in a process called the fetch-execute cycle. As stated earlier, programs reside in memory but are executed in the CPU. Each instruction to be executed must be moved from memory to the processor. The address of the next instruction to be executed is maintained in a register called the program counter. The fetch- execute cycle can be simply described by the following algorithm:

```
initialize the program counter
repeat forever
        fetch the instruction pointed to by the program counter
        increment the program counter to point at the next instruction
        decode the instruction
       execute the instruction
end repeat
```

The “decode the instruction” step in the algorithm means the instruction is examined to determine what action it specifies. Program execution terminates when a stop instruction is encountered, although on an actual computer a stop instruction is rarely executed. Rather, control transfers from the operating system to a user program for its execution and then back to the operating system when the user program execution is complete. In a computer system in which more than one user program may be in memory at a given time, this process is far more complex.

As stated earlier, a functional, or applicative, language is one in which the primary means of computation is applying functions to given parameters. Programming can be done in a functional language without the kind of variables that are used in imperative languages, without assignment statements, and Arithmetic and logic unit Control unit Memory (stores both instructions and data) Instructions and data Input and output devices Results of operations Central processing unit Figure 1.1 The von Neumann computer architecture 1.4 Influences on Language Design 43 without iteration. Although many computer scientists have expounded on the myriad benefits of functional languages, such as Scheme, it is unlikely that they will displace the imperative languages until a non–von Neumann computer is designed that allows efficient execution of programs in functional languages. Among those who have bemoaned this fact, the most eloquent is John Backus (1978), the principal designer of the original version of Fortran.

In spite of the fact that the structure of imperative programming languages is modeled on a machine architecture, rather than on the abilities and inclinations of the users of programming languages, some believe that using imperative languages is somehow more natural than using a functional language. So, these people believe that even if functional programs were as efficient as imperative programs, the use of imperative programming languages would still dominate.

Programming Design Methodologies

The late 1960s and early 1970s brought an intense analysis, begun in large part by the structured-programming movement, of both the software development process and programming language design. An important reason for this research was the shift in the major cost of computing from hardware to software, as hardware costs decreased and programmer costs increased. Increases in programmer productivity were relatively small. In addition, progressively larger and more complex problems were being solved by computers. Rather than simply solving sets of equations to simulate satellite tracks, as in the early 1960s, programs were being written for large and complex tasks, such as controlling large petroleum-refining facilities and providing worldwide airline reservation systems. The new software development methodologies that emerged as a result of the research of the 1970s were called top-down design and stepwise refinement. The primary programming language deficiencies that were discovered were incompleteness of type checking and inadequacy of control statements (requiring the extensive use of gotos). In the late 1970s, a shift from procedure- oriented to data- oriented program design methodologies began. Simply put, data-oriented methods empha - size data design, focusing on the use of abstract data types to solve problems.

Implementation Methods

Two of the primary components of a computer are its internal memory and its processor. The internal memory is used to store programs and data. The processor is a collection of circuits that provides a realization of a set of primitive operations, or machine instructions, such as those for arithmetic and logic operations. In most computers, some of these instructions, which are sometimes called macroinstructions, are actually implemented with a set of instructions called microinstructions, which are defined at an even lower level. Because microinstructions are never seen by software, they will not be discussed further here. The machine language of the computer is its set of instructions. In the absence of other supporting software, its own machine language is the only language that most hardware computers “understand.” Theoretically, a computer could be designed and built with a particular high-level language as its machine language, but it would be very complex and expensive.

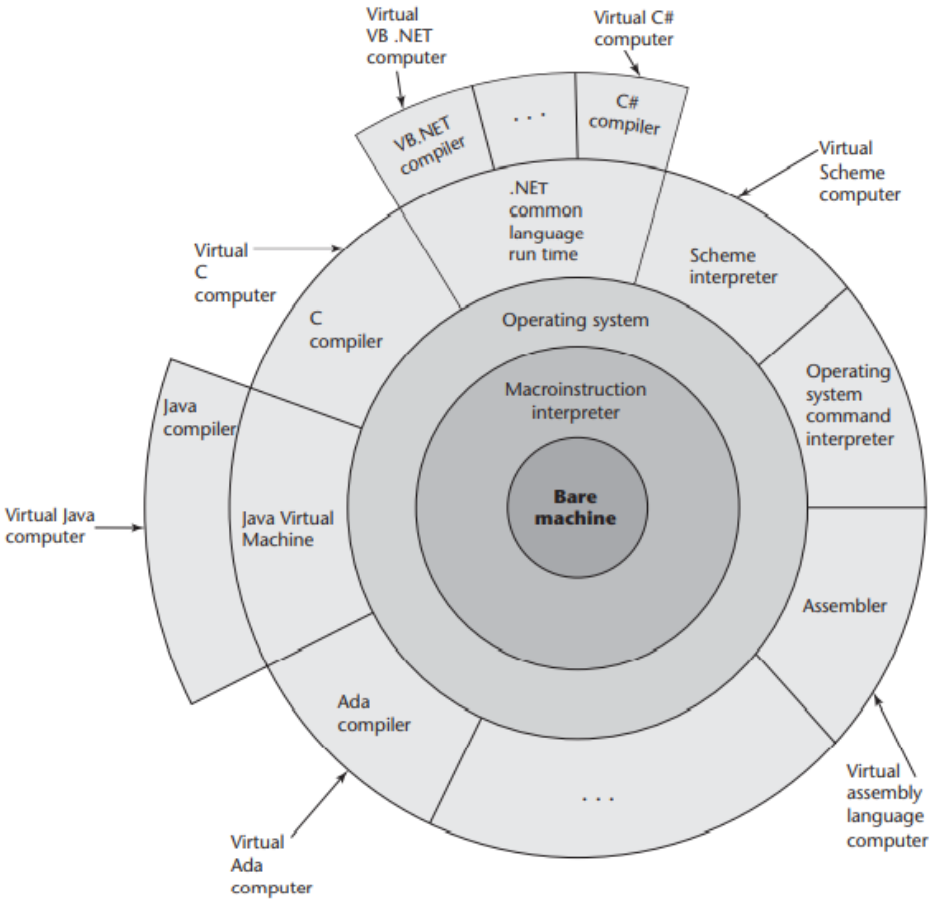
Furthermore, it would be highly inflexible, because it would be difficult (but not impossible) to use it with other high-level languages. The more practical machine design choice implements in hardware a very low- level language that provides the most commonly needed primitive operations and requires system software to create an interface to programs in higher-level languages. A language implementation system cannot be the only software on a computer. Also required is a large collection of programs, called the operating system, which supplies higher-level primitives than those of the machine language. These primitives provide system resource management, input and output operations, a file management system, text and/or program editors, and a variety of other commonly needed functions. Because language implementation systems need many of the operating system facilities, they interface with the operating system rather than directly with the processor (in machine language).

Compilation

Programming languages can be implemented by any of three general methods. At one extreme, programs can be translated into machine language, which can be executed directly on the computer. This method is called a compiler.

Figure 1.2

Layered interface of virtual computers, provided by a typical computer system

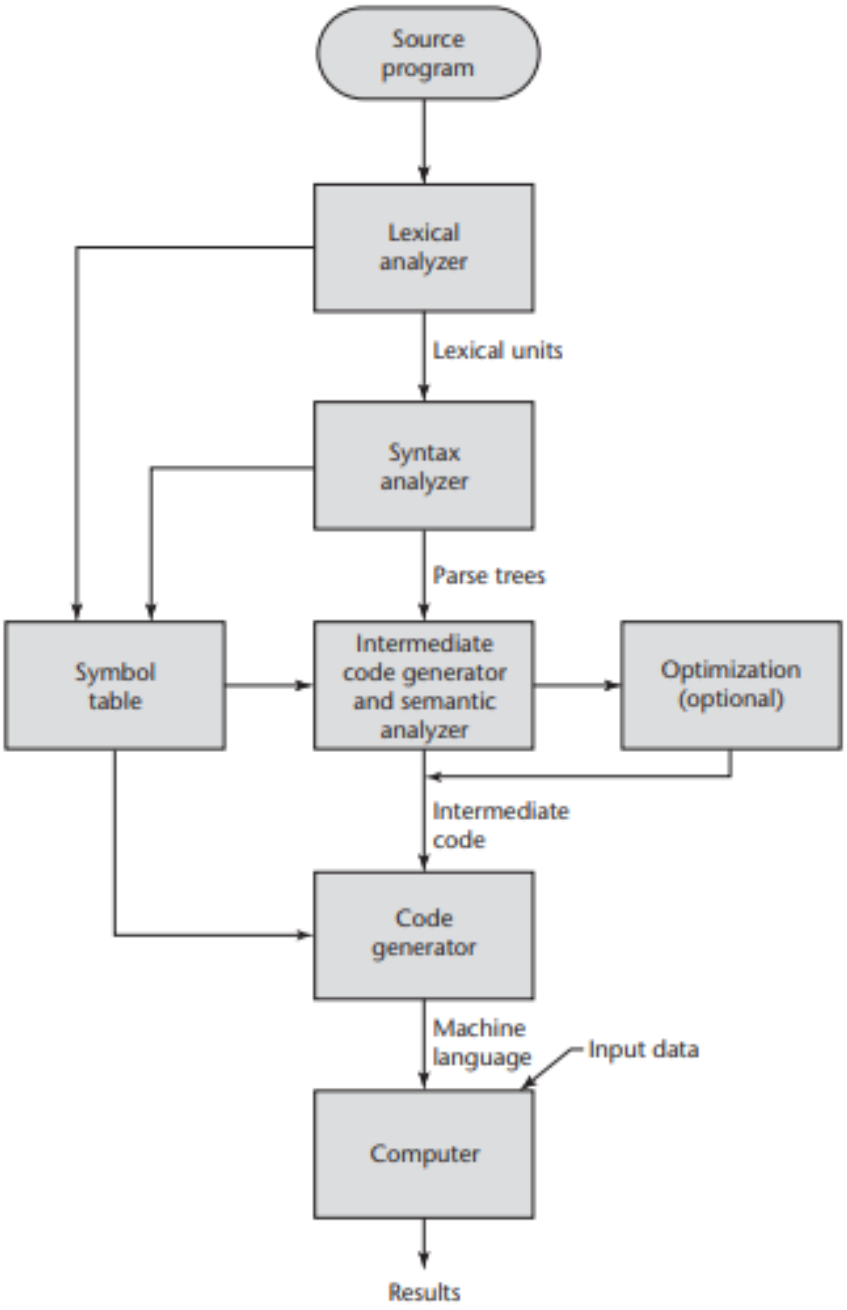


implementation and has the advantage of very fast program execution, once the translation process is complete. Most production implementations of languages, such as C, COBOL, and C++, are by compilers.

The language that a compiler translates is called the source language. The process of compilation and program execution takes place in several phases, the most important of which are shown in Figure 1.3.

The lexical analyzer gathers the characters of the source program into lexical units. The lexical units of a program are identifiers, special words, operators,

Figure 1.3
The compilation process



and punctuation symbols. The lexical analyzer ignores comments in the source program because the compiler has no use for them.

The syntax analyzer takes the lexical units from the lexical analyzer and uses them to construct hierarchical structures called parse trees. These parse trees represent the syntactic structure of the program. In many cases, no actual parse tree structure is constructed; rather, the information that would be required to build a tree is generated and used directly. Both lexical units and parse trees are further discussed in Chapter 3. Lexical analysis and syntax analysis, or parsing.

The intermediate code generator produces a program in a different language, at an intermediate level between the source program and the final output of the compiler: the machine language program.4 Intermediate languages sometimes look very much like assembly languages, and in fact, sometimes are actual assembly languages. In other cases, the intermediate code is at a level somewhat higher than an assembly language. The semantic analyzer is an integral part of the intermediate code generator. The semantic analyzer checks for errors, such as type errors, that are difficult, if not impossible, to detect during syntax analysis.

Optimization, which improves programs (usually in their intermediate code version) by making them smaller or faster or both. Because many kinds of optimization are difficult to do on machine language, most optimization is done on the intermediate code.

The code generator translates the optimized intermediate code version of the program into an equivalent machine language program.

The symbol table serves as a database for the compilation process. The primary contents of the symbol table are the type and attribute information of each user-defined name in the program. This information is placed in the symbol table by the lexical and syntax analyzers and is used by the semantic analyzer and the code generator.

As stated previously, although the machine language generated by a compiler can be executed directly on the hardware, it must nearly always be run along with some other code. Most user programs also require programs from the operating system. Among the most common of these are programs for input and output. The compiler builds calls to required system programs when they are needed by the user program. Before the machine language programs produced by a compiler can be executed, the required programs from the operating system must be found and linked to the user program. The linking operation connects the user program to the system programs by placing the addresses of the entry points of the system programs in the calls to them in the user program. The user and system code together are sometimes called a load module, or executable image. The process of collecting system programs and linking them to user programs is called linking and loading, or sometimes just linking. It is accomplished by a systems program called a linker.

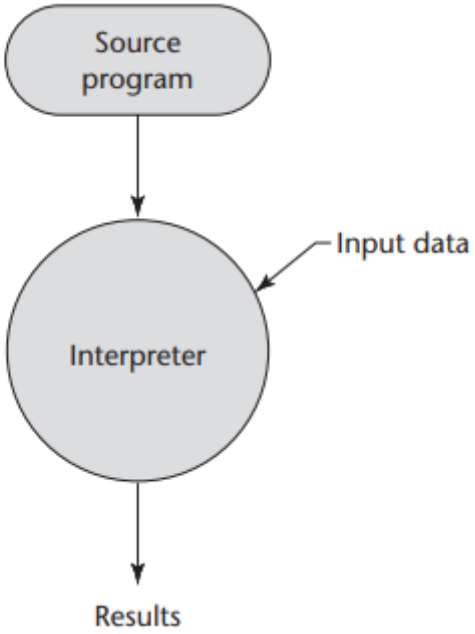
Pure interpretation

Lies at the opposite end (from compilation) among implementation methods. With this approach, programs are interpreted by another program called an interpreter, with no translation whatever. The interpreter program acts as a software simulation of a machine whose fetch-execute cycle deals with high- level language program statements rather than machine instructions. This software simulation obviously provides a virtual machine for the language. Pure interpretation has the advantage of allowing easy implementation of many source-level debugging operations, because all run-time error messages can refer to source- level units. For example, if an array index is found to be out of range, the error message can easily indicate the source line of the error and the name of the array. On the other hand, this method has the serious disadvantage that execution is 10 to 100 times slower than in compiled systems. The primary source of this slowness is the decoding of the high- level language statements, which are far more complex than machine language instructions (although there may be fewer statements than instructions in equivalent machine code). Furthermore, regardless of how many times a statement is executed, it must be decoded every time. Therefore, statement decoding, rather than the connection between the processor and memory, is the bottleneck of a pure interpreter. Another disadvantage of pure interpretation is that it often requires more space. In

addition to the source program, the symbol table must be present during interpretation. Furthermore, the source program may be stored in a form designed for easy access and modification rather than one that provides for minimal size. Although some simple early languages of the 1960s (APL, SNOBOL, and Lisp) were purely interpreted, by the 1980s, the approach was rarely used on high-level languages. However, in recent years, pure interpretation has made a significant comeback with some Web scripting languages, such as JavaScript and PHP, which are now widely used. The process of pure interpretation is shown in Figure 1.4.

Figure 1.4

Pure interpretation



Hybrid Implementation Systems

Some language implementation systems are a compromise between compilers and pure interpreters; they translate high-level language programs to an intermediate language designed to allow easy interpretation. This method is faster than pure interpretation because the source language statements are decoded only once. Such implementations are called hybrid implementation systems.

The process used in a hybrid implementation system is shown in Figure 1.5. Instead of translating intermediate language code to machine code, it simply interprets the intermediate code.

Perl is implemented with a hybrid system. Perl programs are partially compiled to detect errors before interpretation and to simplify the interpreter.

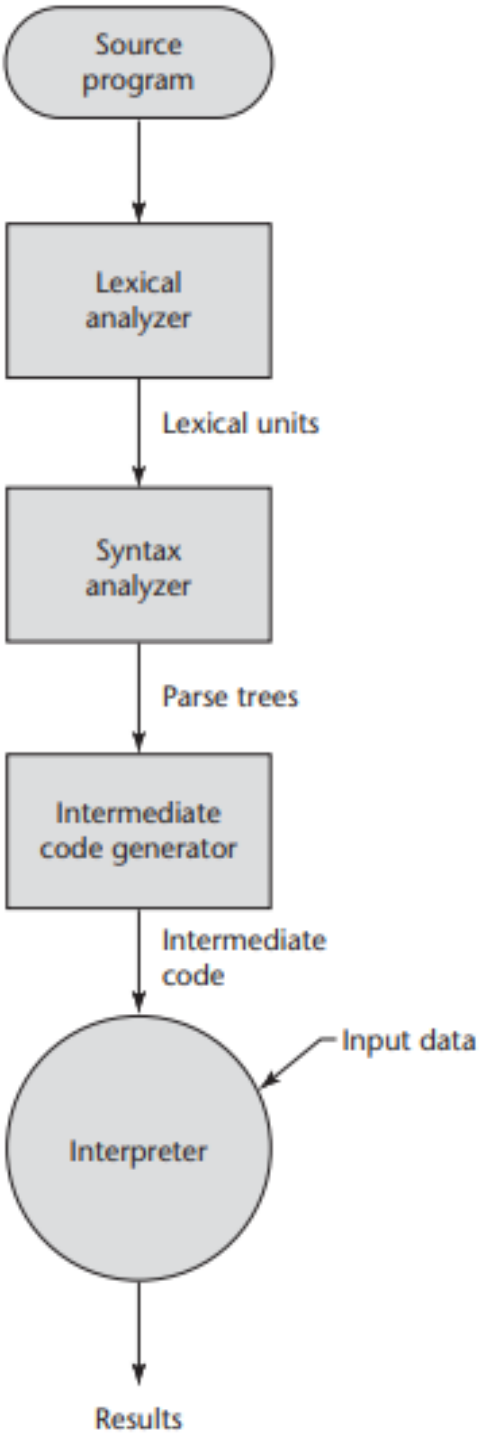
Initial implementations of Java were all hybrid. Its intermediate form, called byte code, provides portability to any machine that has a byte code interpreter and an associated run-time system. Together, these are called the Java Virtual Machine. There are now systems that translate Java byte code into machine code for faster execution.

A Just-in-Time (JIT) implementation system initially translates programs to an intermediate language. Then, during execution, it compiles intermediate language methods into machine code when they are called. The machine code version is kept for subsequent calls. JIT systems now are widely used for Java programs. Also, the .NET languages are all implemented with a JIT system.

Sometimes an implementor may provide both compiled and interpreted implementations for a language. In these cases, the interpreter is used to develop and debug programs. Then, after a (relatively) bug-free state is reached, the programs are compiled to increase their execution speed.

Figure 1.5

Hybrid implementation system



Evolution of Major Programming Language

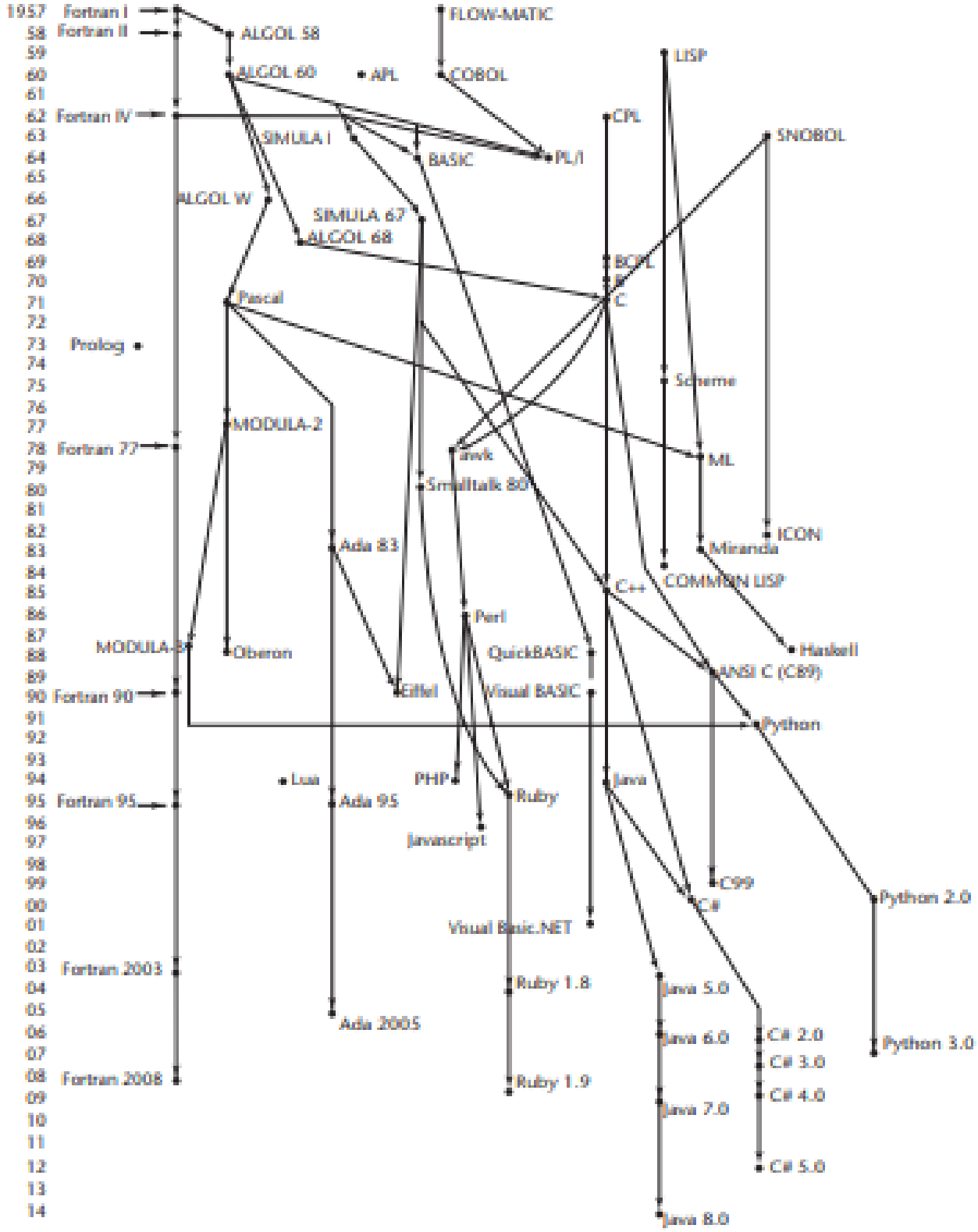


Figure 2.1

1. FORTRAN 90
2. LISP
3. ALGOL 60 - Genealogy
4. COBOL - Genealogy
5. QuickBASIC - Genealogy
6. PL / I - Genealogy
7. Pascal - Genealogy
8. C - Genealogy
9. Ada - Genealogy
10. Smalltalk - Genealogy

Zuse's Plankalkul (program calculus)

The first programming language discussed in this chapter is highly unusual in several respects. For one thing, it was never implemented. Furthermore, although developed in 1945, its description was not published until 1972. Because so few people were familiar with the language, some of its capabilities did not appear in other languages until 15 years after its development. @ A system developed for MIT's whirlwind computer.

Historical Background Between 1936 and 1945, German scientist Konrad Zuse (pronounced "Tsoozuh") built a series of complex and sophisticated computers from electromechanical relays. By early 1945, Allied bombing had destroyed all but one of his latest models, the Z4, so he moved to a remote Bavarian village, Hinterstein, and his research group members went their separate ways. Working alone, Zuse embarked on an effort to develop a language for expressing computations for the Z4, a project he had begun in 1943 as a proposal for his Ph.D. dissertation. He named this language Plankalkül, which means program calculus. In a lengthy manuscript dated 1945 but not published until 1972 (Zuse, 1972), Zuse defined Plankalkül and wrote algorithms in the language to solve a wide variety of problems Zuse's Plankalkul.

Pseudocodes

First, note that the word pseudocode is used here in a different sense than its contemporary meaning. We call the languages discussed in this section pseudocodes because that's what they were named at the time they were developed and used (the late 1940s and early 1950s). However, they are clearly not pseudocodes in the contemporary sense. The computers that became available in the late 1940s and early 1950s were far less usable than those of today. In addition to being slow, unreliable, expensive, and having extremely small memories, the machines of that time were difficult to program because of the lack of supporting software. There were no high-level programming languages or even assembly languages, so programming was done in machine code, which is both tedious and error prone. Among its problems is the use of numeric codes for specifying instructions. For example, an ADD instruction might be specified by the code 14 rather than a connotative textual name, even if only a single letter. This makes programs very difficult to read. A more serious problem is absolute addressing, which makes program modification tedious and error prone. For example, suppose we have a machine language program stored in memory. Many of the instructions in such a program refer to other locations within the program, usually to reference data or to indicate the targets of branch instructions. Inserting an instruction at any position in the program other than at the end invalidates the correctness of all instructions that refer to addresses beyond the insertion point, because those addresses must be increased to make room for the new instruction. To make the addition correctly, all instructions that refer to addresses that follow the addition must be found and modified. A similar problem occurs with deletion of an instruction. In this case, however, machine

languages often include a "no operation" instruction that can replace deleted instructions, thereby avoiding the problem.

Short Code

The first of these new languages, named Short Code, was developed by John Mauchly in 1949 for the BINAC computer, which was one of the first successful stored-program electronic computers. Short Code was later transferred to a UNIVAC I computer (the first commercial electronic computer sold in the United States) and, for several years, was one of the primary means of programming those machines. Although little is known of the original Short Code because its complete description was never published, a programming manual for the UNIVAC I version did survive (Remington-Rand, 1952). It is safe to assume that the two versions were very similar.

The words of the UNIVAC I's memory had 72 bits, grouped as 12 six-bit bytes. Short Code consisted of coded versions of mathematical expressions that were to be evaluated. The codes were byte-pair values, and many equations could be coded in a word. The following operation codes were included:

01 -	06	abs value	1n (n+2)nd power
02)	07	+	2n (n+2)nd root
03 =	08	pause	4n if <= n
04 /	09	(	58 print and tab

Variables were named with byte-pair codes, as were locations to be used as constants. For example, X0 and Y0 could be variables. The statement

X0 = SQRT(ABS(Y0))

would be coded in a word as 00 X0 03 20 06 Y0. The initial 00 was used as padding to fill the word. Interestingly, there was no multiplication code; multiplication was indicated by simply placing the two operands next to each other, as in algebra. Short Code was not translated to machine code; rather, it was implemented with a pure interpreter. At the time, this process was called automatic programming. It clearly simplified the programming process, but at the expense of execution time. Short Code interpretation was approximately 50 times slower than machine code

The IBM 704 Fortran

Certainly one of the greatest single advances in computing came with the introduction of the IBM 704 in 1954, in large measure because its capabilities prompted the development of Fortran. One could argue that if it had not been IBM with the 704 and Fortran, it would soon thereafter have been some other organization with a similar computer and related high-level language. However, IBM was the first with both the foresight and the resources to undertake these developments.

Functional Programming Lisp

The first functional programming language was invented to provide language features for list processing, the need for which grew out of the first applications in the area of artificial intelligence (AI).

Data Structures

Pure Lisp has only two kinds of data structures: atoms and lists. Atoms are either symbols, which have the form of identifiers, or numeric literals. The concept of storing symbolic information

in linked lists is natural and was used in IPL-II. Such structures allow insertions and deletions at any point, operations that were then thought to be a necessary part of list processing. It was eventually determined, however, that Lisp programs rarely require these operations.

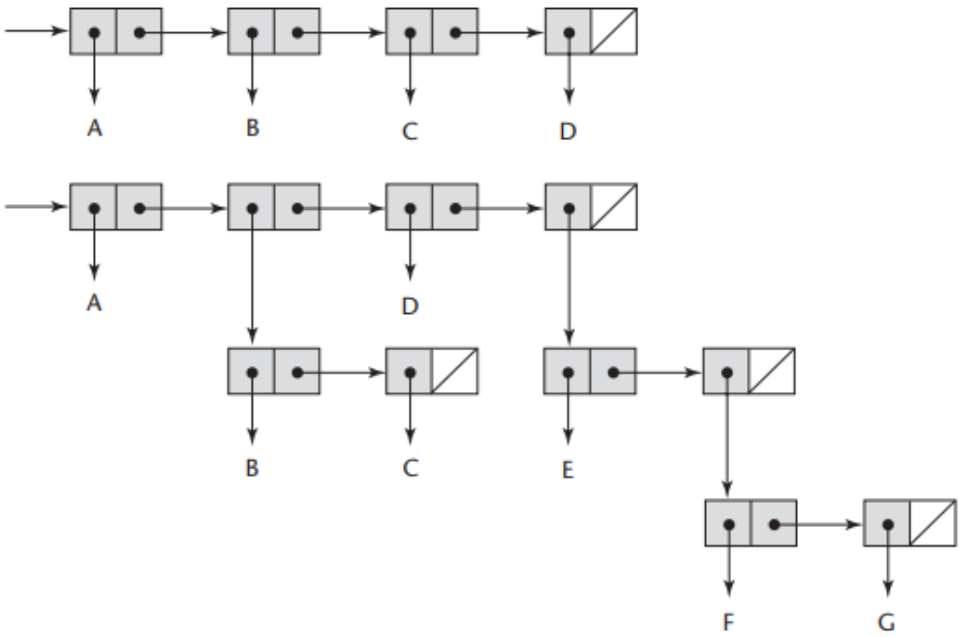
Lists are specified by delimiting their elements with parentheses. Simple lists, in which elements are restricted to atoms, have the form

Processes in Functional Programming

Lisp was designed as a functional programming language. All computation in a purely functional program is accomplished by applying functions to arguments. Neither the assignment statements nor the variables that abound in imperative language programs are necessary in functional language programs. Furthermore, repetitive processes can be specified with recursive function calls, making iteration (loops) unnecessary. These basic concepts of functional programming make it significantly different from programming in an imperative language.

Figure 2.2

Internal representation of two Lisp lists



The Syntax of Lisp

Lisp is very different from the imperative languages, both because it is a functional programming language and because the appearance of Lisp programs is so different from those in languages like Java or C++. For example, the syntax of Java is a complicated mixture of English and algebra, while Lisp's syntax is a model of simplicity. Program code and data have exactly the same form: parenthesized lists. Consider again the list.

(A B C D)

When interpreted as data, it is a list of four elements. When viewed as code, it is the application of the function named A to the three parameters B, C, and D.

The First Step Toward Sophistication: ALGOL 60

Historical Background

ALGOL 60 was the result of efforts to design a universal programming language for scientific applications. By late 1954, the Laning and Zierler algebraic system had been in operation for over a year, and the first report on Fortran had been published. Fortran became a reality in 1957, and several other high-level languages were being developed. Most notable among them were IT, which was designed by Alan Perlis at Carnegie Tech, and two languages for the UNIVAC computers, MATH-MATIC and UNICODE. The proliferation of languages made program sharing among users difficult. Furthermore, the new languages were all growing up around single architectures, some for UNIVAC computers and some for IBM 700-series machines. In response to this blossoming of machine-dependent languages, several major computer user groups in the United States, including SHARE (the IBM scientific user group) and USE (UNIVAC Scientific Exchange, the large-scale UNIVAC scientific user group), submitted a petition to the Association for Computing Machinery (ACM) on May 10, 1957, to form a committee to study and recommend action to create a machine-independent scientific programming language. Although Fortran might have been a candidate, it could not become a universal language, because at the time it was solely owned by IBM. Previously, in 1955, GAMM (a German acronym for Society for Applied Mathematics and Mechanics) had formed a committee to design one universal, machine-independent algorithmic language. The desire for this new language was in part due to the Europeans' fear of being dominated by IBM. By late 1957, however, the appearance of several high-level languages in the United States convinced the GAMM subcommittee that their effort had to be widened to include the Americans, and a letter of invitation was sent to ACM. In April 1958, after Fritz Bauer of GAMM presented the formal proposal to ACM, the two groups officially agreed to a joint language design project.

The following is an example of an ALGOL 60 program:

```
comment ALGOL 60 Example Program
Input: An integer, listlen, where listlen is less than
    100, followed by listlen-integer values
Output: The number of input values that are greater than
    the average of all the input values ;
begin
    integer array intlist [1:99];
    integer listlen, counter, sum, average, result;
    sum := 0;
    result := 0;
```

```
    readint (listlen);
if (listlen > 0) ^ (listlen < 100) then
    begin
    comment Read input into an array and compute the average;
    for counter := 1 step 1 until listlen do
    begin
    readint (intlist[counter]);
    sum := sum + intlist[counter]
    end;
comment Compute the average;
    average := sum / listlen;
    comment Count the input values that are > average;
        for counter := 1 step 1 until listlen do
            if intlist[counter] > average
        then result := result + 1;
comment Print result;
printstring("The number of values > average is:");
printint (result)
    end
else
    printstring ("Error-input list length is not legal";
end
```

The Beginning of Time-Sharing: Basic

Basic (Mather and Waite, 1971) is another programming language that has enjoyed widespread use but has gotten little respect. Like COBOL, it has largely been ignored by computer scientists. Also, like COBOL, in its earliest versions it was inelegant and included only a meager set of control statements. Basic was very popular on microcomputers in the late 1970s and early 1980s. This followed directly from two of the main characteristics of early versions of Basic. It was easy for beginners to learn, especially those who were not science oriented, and its smaller dialects could be implemented on computers with very small memories.6 When the capabilities of microcomputers grew and other languages were implemented, the use of Basic waned. A strong resurgence in the use of Basic began with the appearance of Visual Basic (Microsoft, 1991) in the early 1990s.

Everything for PL/1

PL/I represents the first large-scale attempt to design a language that could be used for a broad spectrum of application areas. All previous and most subsequent languages have focused on one particular application area, such as science, artificial intelligence, or business.

More History of Programming Languages and Programming Samples

Let's Remember:

In this chapter we introduced the study of programming language design and implementation. We considered why there are so many languages, what makes them successful or unsuccessful, how they may be categorized for study, and what benefits the reader is likely to gain from that study. We noted that language design and language implementation is intimately tied to one another. Obviously an implementation must conform

to the rules of the language. At the same time, a language designer must consider how easy or difficult it will be to implement various features, and what sort of performance is likely to result. Language implementations are commonly differentiated into those based on interpretation and those based on compilation. We noted, however, that the difference between these approaches is fuzzy, and that most implementations include a bit of each. As a general rule, we say that a language is compiled if execution is preceded by a translation step that (1) fully analyzes both the structure (syntax) and meaning (semantics) of the program, and (2) produces an equivalent program in a significantly different form. The bulk of the implementation material in this book pertains to compilation. Compilers are generally structured as a series of phases. The first few phases—scanning, parsing, and semantic analysis—serve to analyze the source program. Collectively these phases are known as the compiler's front end. The final few phases—target code generation and machine-specific code improvement—are known as the back end. They serve to build a target program—preferably a fast one—whose semantics match those of the source. Between the front end and the back end, a good compiler performs extensive machine-independent code improvement; the phases of this "middle end" typically comprise the bulk of the code of the compiler, and account for most of its execution time and from the point of view of the language implementer.

Let's Do This:

1. What was the first programming language you learned? If you chose it, why did you do so? If it was chosen for you by others, why do you think they chose it? What parts of the language did you find the most difficult to learn?
2. Which programming language for scientific applications was the first to be used successfully?
3. In which programming language were most of the AI applications developed prior to 1990?
4. Write sample program on each programming language below:
    1. FORTRAN 90
    2. LISP
    3. ALGOL 60 - Genealogy

Let's Check:

No answer key is provided since the assessment is an essay type.

Suggested Readings:

Sebesta, Robert W., Concepts of Programming Languages 11th Ed, 2016.

Module Post Test

1. What are the Preliminaries?
2. What are the reasons for studying concepts of programming languages?
3. What was the simplest data type in Plankalkül?
4. Describe the programming domains.
5. Why is it that the computer architecture of John von Neumann is very effective?
6. Differentiate Compiler and Interpreter?
7. Discuss the compilation process of computer program statements.

8. Write sample program on each programming language below:
    1. COBOL - Genealogy
    2. QuickBASIC - Genealogy
    3. PL / I - Genealogy
    4. Pascal - Genealogy
    5. C - Genealogy
    6. Ada - Genealogy
    7. Smalltalk - Genealogy

References/Sources:

Sebesta, Robert W., Concepts of Programming Languages 11th Ed, 2016.

Scott, Michael L.. *Programming Language Pragmatic 4th Edition*. Morgan Kaufmann, 2015.

Krishnamurthi, Shriram. Programming Languages: Application and Interpretation 2nd Edition Self Published, 2017.

Mitchell, John C. Concepts in Programming Languages, Cambridge University Press, 1988.